

Diffusion

1. 들어가며

이번 글에서는 Diffusion Model의 기초부터 응용까지 모든 내용을 살펴보겠습니다. 크게 두 챕터로 구성되어 있는데요.

첫 번째 챕터에서는 Diffusion Model의 기초에 대해 살펴봅니다. 먼저 Image Generative Model의 한 종류로써 Diffusion Model이 무엇을 할 수 있는지 살펴봅니다. 다음으로 Diffusion Model의 작동 원리를 자세히 살펴볼거고요. 이렇게 동작이 가능하도록 하는 Diffusion Model 공통적으로 사용되는 Architecture와 Loss Function에 대해 알아봅니다.

두 번째 챕터에서는 Diffusion Model의 응용 버전에 대해 살펴봅니다. 먼저 특정 Class 이미지만 생성하는 방법을 알아봅니다. 이어서 Super-Resolution과 Inpainting을 위한 Diffusion Model을 살펴보고요. 다음으로 이미지를 묘사하는 Text에 맞게 이미지를 생성하는 방법, 다양한 형태의 입력을 받는 방법, 동시에 다양한 형태의 입력을 받는 방법을 차례대로 살펴보겠습니다.

2. Diffusion Model이란?

이번 챕터에서는 Diffusion Model의 기초를 다져봅니다. Diffusion Model이 무엇인지, 어떻게 동작하는지를 자세히 살펴보겠습니다.

2-1. Image Generative Model

Diffusion Model은 이미지 생성 모델 (Image Generative Model)의 일종입니다. 일반적으로 가장 많이 알려진 이미지 생성 모델로는 GAN이 있습니다. GAN은 이미지를 생성하는 Generator와 이를 구분하는 Discriminator가 상호 적대적으로 학습하는 방법을 사용합니다. Diffusion Model은 원리는 다르지만 GAN과 마찬가지로 이미지를 생성할 수 있는 모델인데요. GAN과 Diffusion Model이 생성한 이미지를 비교해보겠습니다.

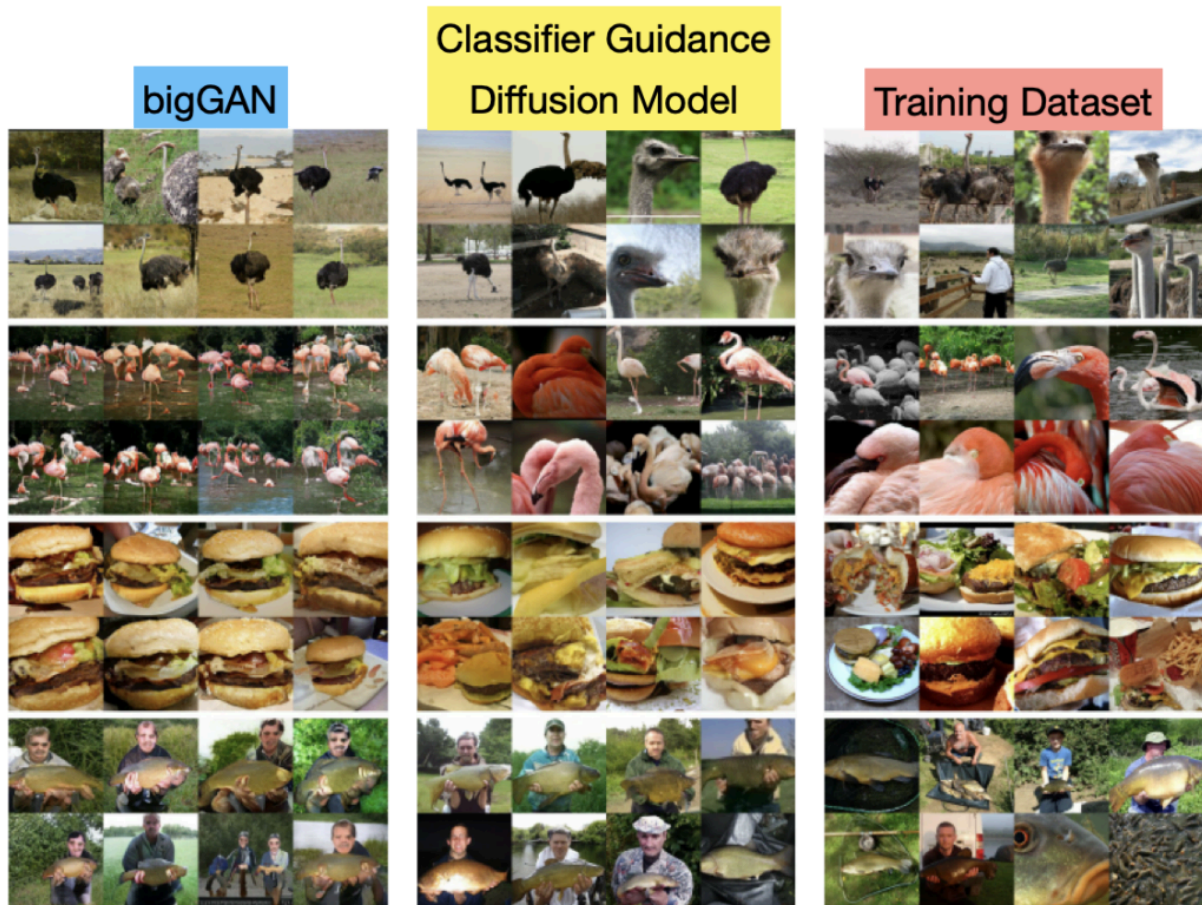


그림1. GAN과 Diffusion Model

위 그림은 GAN의 대표 모델인 bigGAN과 Classifier Guidance 방식의 Diffusion Model이 생성한 클래스 별 이미지를 비교한 사진입니다. 가장 오른쪽은 클래스별 학습 데이터이고요. 왼쪽은 bigGAN이 생성한 이미지를, 가운데는 Diffusion Model이 생성한 이미지를 보여주고 있습니다. 어떨까요? GAN과 비교했을때 생성한 이미지의 퀄리티가 전혀 떨어지지 않죠?

하지만 Diffusion Model의 진짜 능력은 단순히 이미지를 생성 하는데에 있지 않습니다. 아래 그림을 한번 볼게요.



a teddy bear on a skateboard in times square

그림2. DALLE2가 생성한 이미지

타임 스퀘어에서 스케이트 보드를 타고 있는 테디 베어 사진인데요. DALLE2 라는 Diffusion Model이 생성한 이미지입니다. 만들고자 하는 이미지를 설명하는 Text를 입력 받아 이에 해당하는 이미지를 생성해주는 모습인데요. 놀라운건 저렇게 타임 스퀘어에서 스케이트 보드를 타는 테디 베어 사진은 DALLE2가 학습한 데이터셋에는 없다는 점입니다. DALLE2는 학습 데이터에 없는 형태의 이미지들도 입력으로 주어진 Text가 묘사하는 대로 이미지를 생성할 수 있죠.

한 스텝 더 나아가 볼까요? 이번에는 이미지를 설명하는 Text 뿐만 아니라 위치 정보까지 입력으로 넣어서 이미지를 생성해보겠습니다.

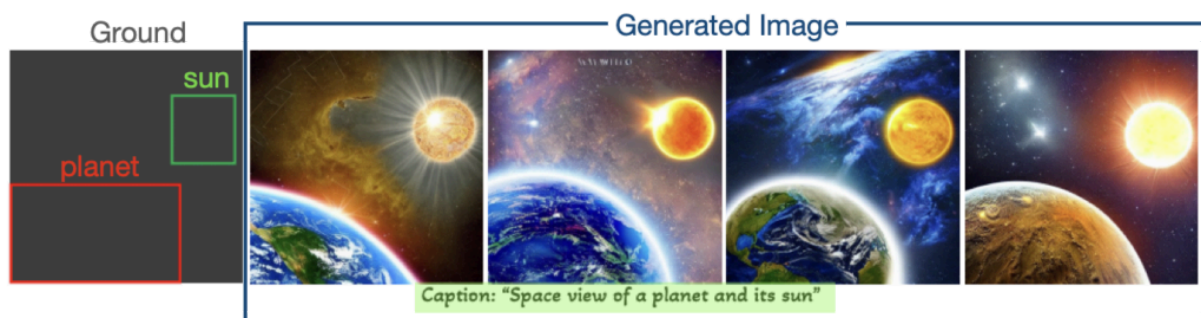


그림3. GLIGEN이 생성한 이미지

위 그림은 GLIGEN이라는 Diffusion Model이 생성한 이미지 샘플인데요. 그림 아래에 생성하고자 하는 이미지를 설명하는 Text인 Caption이 있습니다. 행성과 태양의 우주 모습을 그리고 싶은데요. 왼쪽의 Ground는 이때의 행성과 태양의 위치를 제한해주고 있습니다. 즉 왼쪽 아래에는 행성이, 오른쪽 위에는 태양이 위치하도록 우주 모습을 그려달라는 것이죠. 오른쪽 네개 이미지는 이러한 조건을 입력 받아 GLIGEN이 생성한 이미지들입니다. 주어진 조건을 잘 만족하면서도 다양한 형태로 잘 만들어진 모습이죠?

이렇게 기존 Image Generative Model과는 달리 다양한 요구조건에 맞는 고화질 이미지를 생성할 수 있는 모습을 보여주면서 최근 Diffusion Model이 큰 화제가 되고 있습니다.

2-2. Diffusion Model 작동 원리

이러한 엄청난 능력의 Diffusion Model은 어떻게 동작 하는걸까요?

2-2-1. Diffusion Process

Diffusion Model에 대해 알려면 먼저 Diffusion 이 무엇인지 알아야겠죠. Diffusion은 '확산' 현상을 의미하는데요.

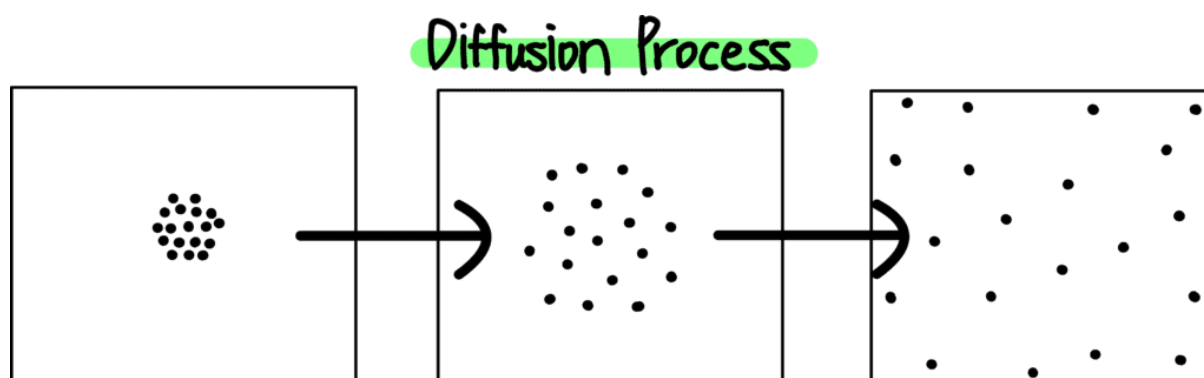


그림4. Forward Diffusion Process

위 그림은 공간상에 모여 있던 분자들이 확산 현상을 통해 전 공간에 고르게 분포하게 되는 현상을 표현하고 있습니다. 이때 분자 하나 하나는 어떻게 움직였을까요? 위 그림의 분자들 중 하나의 분자에만 초점을 맞춰 아주 짧은 시간 단위로 스냅샷을 찍어 관찰했다고 상상해보죠. 그리고 그 분자는 아래와 같이 움직였다고 가정해보겠습니다.

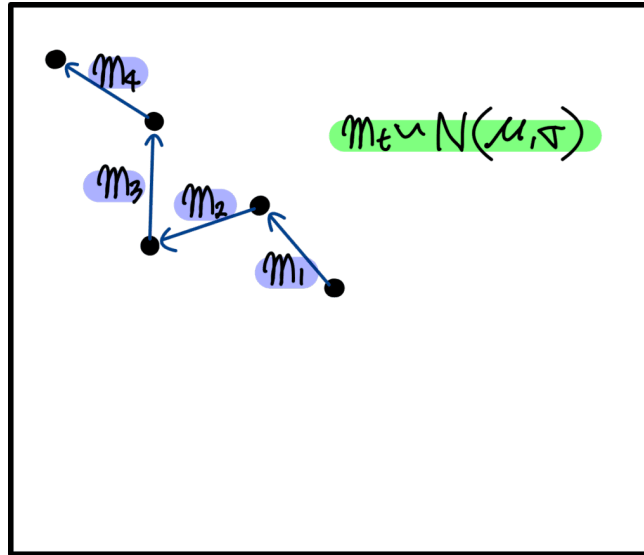


그림5. 분자 하나의 움직임

이러한 분자의 움직임은 Gaussian Distribution을 따르는데요. 즉 위의 그림과 같이 평균 μ 과 표준 편차 σ 를 따르는 정규 분포로 움직임을 표현할 수 있죠. 그렇다면 말이죠, 매시간 t 마다 각 분자의 움직임을 계산해낼 수 있다면 어떻게 될까요? 그럼 Diffusion Process를 거꾸로 되돌릴 수 있겠죠? 아래 그림처럼요.

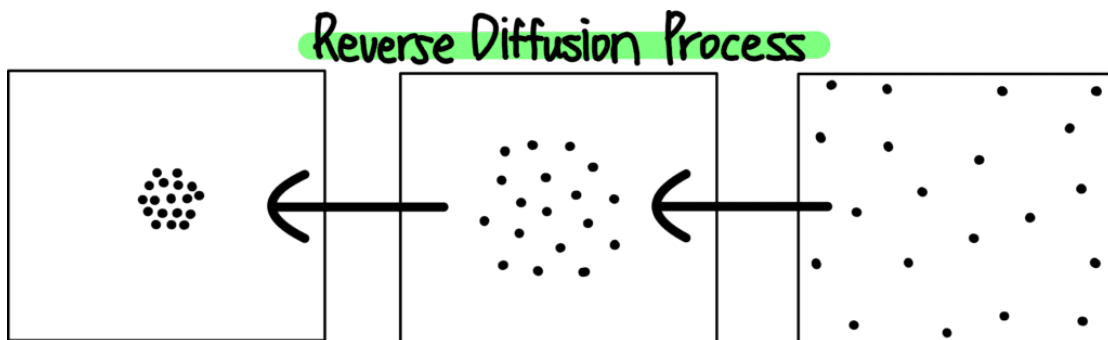


그림6. Reverse Diffusion Process

2-2-2. Image에 Diffusion Process 적용하기

자, 지금까지의 과정을 그대로 Image의 Pixel 값에 적용해보겠습니다. 분자의 움직임을 정규 분포를 따르는 Noise로 보았잖아요. 마찬가지로 Image의 Pixel 값에 정규 분포를 따르는 Noise 값을 추가한다고 생각해보겠습니다.

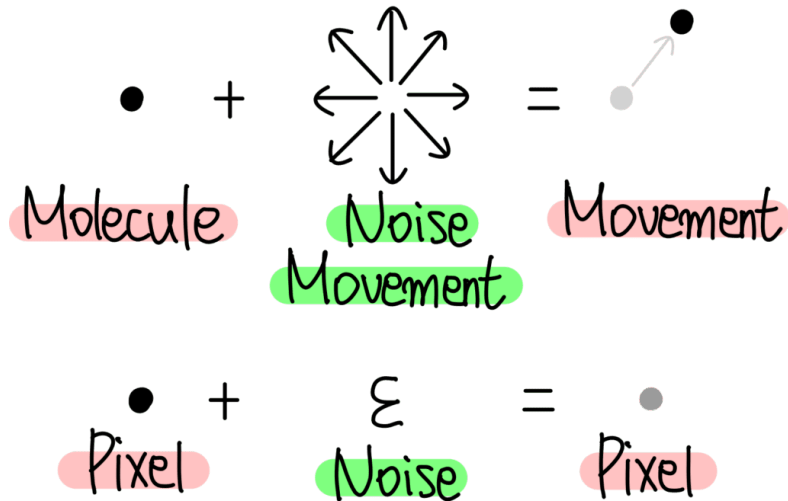


그림7. 이미지 픽셀에 Diffusion 적용하기

그렇다면 Diffusion Process는 이런 형태로 표현되겠네요.

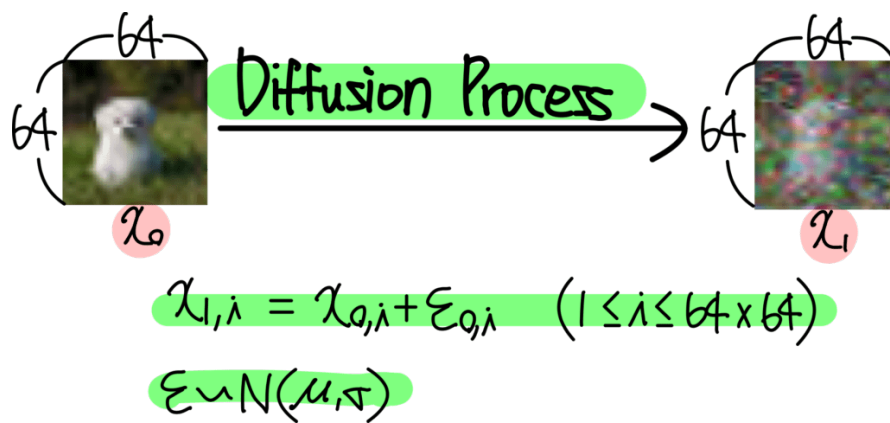


그림8. Image에 적용한 Diffusion

왼쪽에 있는 그림은 원본 이미지입니다. x_0 라고 표현하고 있죠. 이 이미지의 모든 픽셀값에 Noise를 추가해줍니다. 이렇게 만들어진 이미지가 x_1 입니다. 분자의 Diffusion Process와 동일한 방법을 하고 있죠? 이제 이 과정을 매우 많이 반복해야겠죠. 분자들이 아주 많은 시간 t 를 거쳐 최종적으로 완전 균등한 상태로 분포하게 된 것 처럼 말이죠. 그럼 이 과정은 이렇게 표현될겁니다.

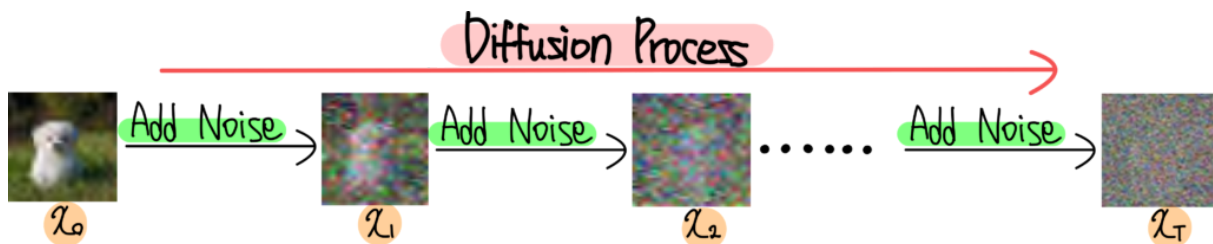


그림9. 이미지의 Forward Diffusion Process

이 과정은 Diffusion Process의 Forward 과정이라고 말할 수 있습니다. 이미지 x_0 로부터 모든 픽셀값에 시간 t 마다 Noise를 첨가해주는 Process죠. 이렇게 아주 긴 시간 T 번을 거쳐 만들어진 이미지 x_T 는 이제 완전한 Noise 이미지가 되어 있겠죠. 분자들의 Diffusion Process에서는 x_0 가 뭉쳐있는 상태, x_T 가 완전히 균등하게 분포하고 있는 상태에 해당합니다.

자, 아까 매시간 t 에서 첨가된 Noise를 계산할 수 있다면 이 과정을 되돌릴 수 있다고 했었죠? 분자들의 경우에는 모든 공간에 균등하게 분포하는 상태에서 다시 한 공간에 모여있는 상태로 되돌릴 수 있는거죠. 그렇다면 이미지의 경우에는 어떻게 될까요?

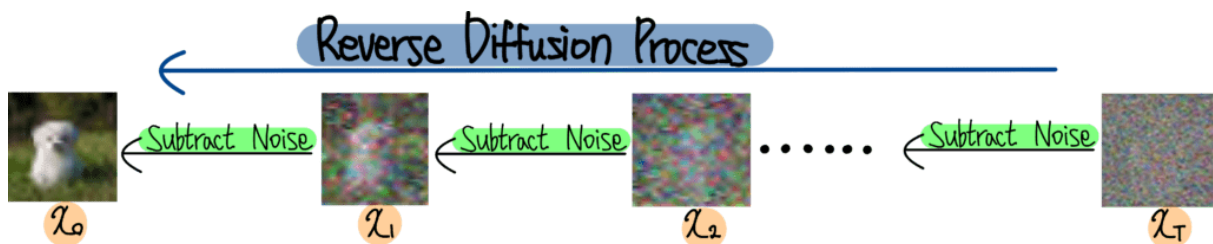


그림10. 이미지의 Reverse Diffusion Process

위와 같은 Process가 될 겁니다. 시간 t 마다 이미지 픽셀값에 첨가된 Noise 값을 계산할 수 있다면 Noise 이미지 x_T 로부터 진짜 이미지 x_0 로 되돌리는 것이 가능하겠죠. 이 말은 곧 이미지를 생성할 수 있다는 말이잖아요? 입력으로는 랜덤하게 생성해준 Noise 이미지 x_T 가 들어가면 Reverse Diffusion Process를 거쳐 완전한 이미지 x_0 가 나오게 되니까 말이죠.

2-3. Architecture

자, 그럼 Diffusion Model에게 필요한 것은 t 시점의 이미지를 입력으로 받아 각 픽셀별로 추가된 Noise를 예측해주는 것이겠군요. 이렇게 계산해낸 Noise를 빼내면 한 시점 전 이미지로, 즉 덜 Noisy한 이미지로 바꿔줄 수 있는거죠. 이 과정을 계속해 나가면 완전한 이미지 x_0 를 만들어낼 수 있을겁니다.

따라서 Diffusion Model이 갖춰야 하는 몇 가지 조건을 정리해보면 다음과 같습니다.

첫 번째로 Input은 이미지, 또는 Noisy 이미지가 될 겁니다. 꼭 이미지여야만 한다는 뜻이 아니라 Width*Height*Channel 형태의 3차원 또는 2차원 Array 여야 한다는 것이죠.

두 번째로 몇 번째 Process 인지를 의미하는 t 도 주어져야 합니다. 그래야 Diffusion Model 입장에서는 그에 맞게 Noise를 예측할 수 있으니까요.

세 번째로 추가 Condition (조건) 이 있다면 이 조건 또한 Diffusion Model에게 주어져야 합니다. 이때의 조건이란 특정 클래스 정보, 또는 생성한 이미지를 표현할 Text 정보 등이 해당합니다.

네 번째로 Output은 Input과 동일한 형태여야 합니다. 대신 값은 각 픽셀별로 첨가된 Noise 값을 의미해야겠죠. Diffusion Model에 따라 Noise 값 자체를 예측할지, Noise의 μ, σ 등을 예측할지는 조금씩 다르지만, Noise를 예측한다고 이해하면 큰 흐름을 이해하기에 무방합니다.

따라서 이를 만족하는 Diffusion Model의 Architecture는 다음과 같습니다.

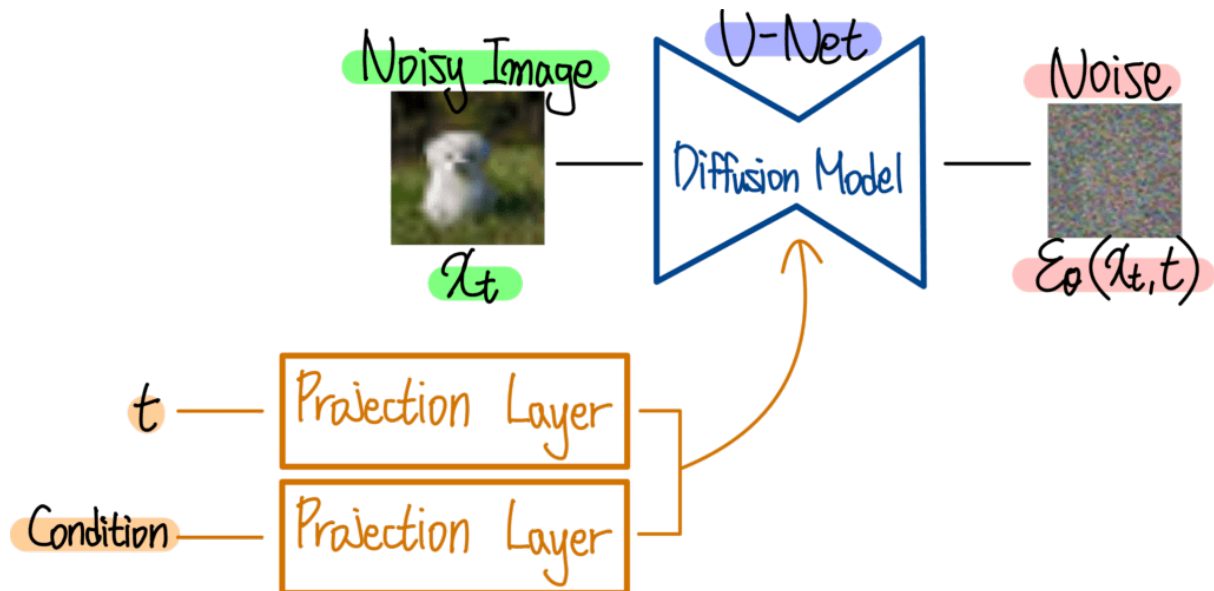


그림11. Diffusion Model Architecture

위 그림은 Diffusion Model의 Architecture를 표현한 그림인데요. 먼저 Diffusion Model은 UNet 구조를 하고 있습니다. 이는 Input과 동일한 Resolution의 Output을 내기에 적절한 구조죠. 그리고 t와 Condition을 별도로 입력받는 모습을 볼 수 있습니다. 이러한 구조는 거의 대부분의 Diffusion Model에서 공통적으로 적용하고 있습니다.

2-4. Loss Function

다음으로 Loss Function을 살펴보겠습니다. 위의 내용까지 이해했다면 Loss Function은 어렵지 않게 생각해낼 수 있는데요. Diffusion Model은 각 픽셀마다 첨가된 Noise를 예측한다고 했잖아요? 따라서 Loss Function은 다음과 같이 표현됩니다.

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \text{실제 Noise } \epsilon - \text{모델이 예측한 Noise } \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

그림12. Diffusion Model Loss Function

ε 은 실제 Noise, 즉 Ground Truth에 해당하고요. ε_{θ} 는 Diffusion Model이 예측한 Noise에 해당합니다. 이 둘이 같아지도록 Diffusion Model을 학습해줍니다.

3. Diffusion Model 응용하기

지금까지 Diffusion Model의 기초에 대해 살펴보았습니다. Diffusion Model이 무엇인지, 무엇을 할 수 있는지, 어떻게 동작하는지 살펴보았죠. 이번 챕터에서는 Diffusion Model의 응용 버전에 대해 알아보겠습니다. 이전 챕터에서 Diffusion Model은 이미지를 생성할 수 있는 모델이라고 했는데요. 이게 끝이 아닙니다. 단순히 이미지만 생성할 수 있는 정도를 넘어서서 아주 다채롭게 활용이 가능하죠. 하나씩 살펴보겠습니다.

3-1. Class Guided Image Generation

첫 번째 응용 버전은 Class Guided Image Generation 인데요. 이는 내가 원하는 Class의 이미지만 생성하는 방법을 의미합니다. 예를 들어 ImageNet 데이터셋을 학습한 Diffusion Model이 있다고 가정해보죠. ImageNet에는 1000개의 클래스가 존재하잖아요. 따라서 이렇게 학습한 Diffusion Model에게 이미지를 생성하라고 시키면 1000가지 클래스의 이미지를 생성하겠죠. 그렇다면 내가 원하는 클래스의 이미지만 생성할 수 있을까요? 이를 위한 대표적인 방법 두 가지가 있는데요. 첫 번째 방법은 Classifier Guidance 방법입니다. 2022년 NIPS에 발표된 Diffusion Models Beat GANs on Image Synthesis 논문에서 제안한 방법이죠. 구조는 다음과 같습니다.

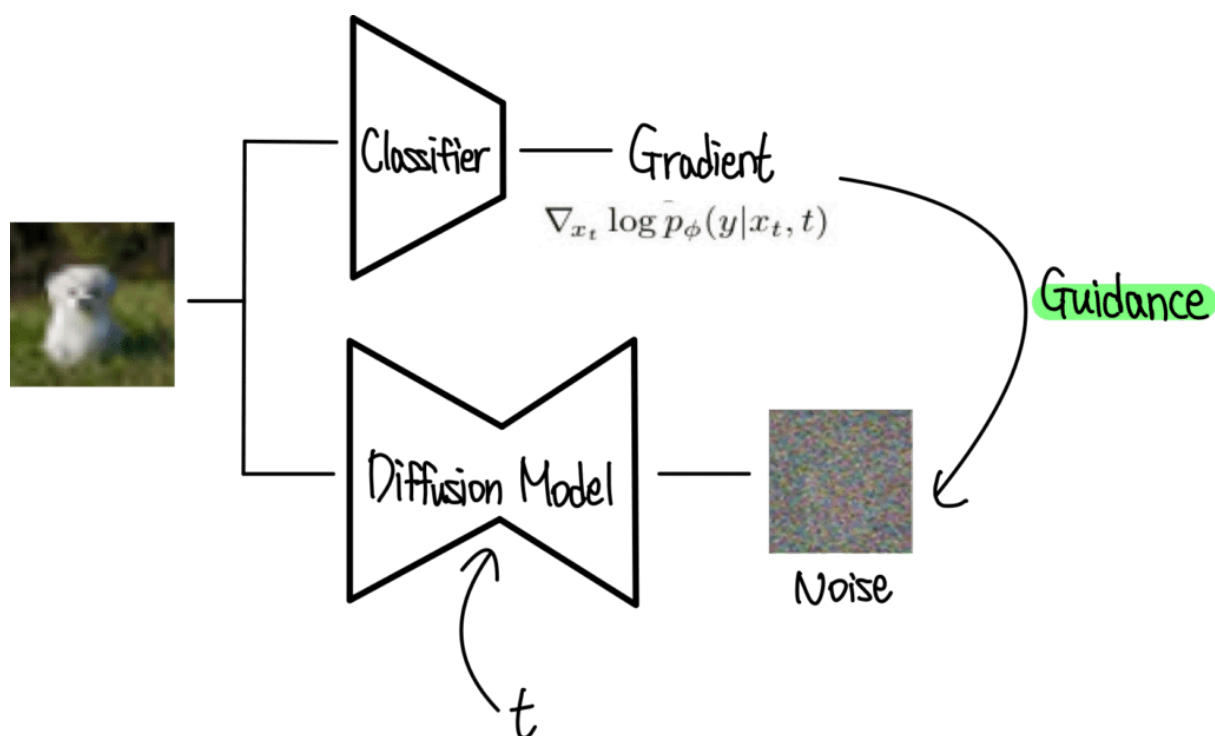


그림13. Classifier Guidance

먼저 Noisy Image를 사용해서 Classifier를 학습해 줍니다. 그리고 나서 이렇게 학습된 Classifier로부터 나오는 Gradient가 Diffusion Model이 학습하는 과정에서 Guidance 역할을 해주는 것이죠. 예를 들어 강아지 이미지를 생성하고 싶다면 Diffusion Model이 강아지가 아닌 이미지를 생성하려고 할 때 마다 큰 Gradient를 보내서 방향을 틀어주는 방법 인거죠. 자세한 내용은 [Classifier Guidance 설명글](#)을 참고해 주세요.

두 번째 방법은 [Classifier Free Guidance](#) 방법입니다. 이 방법은 2021년 NIPS Workshop에 발표된 Classifier-free diffusion guidance 논문에서 제안한 방법인데요. 아무래도 Classifier Guidance는 Classifier를 별도로 학습해야 하기에 귀찮은 방법이죠. 이에 Classifier Free Guidance에서는 Classifier 없이 한번에 Guidance 할 수 있는 방법을 제안합니다.

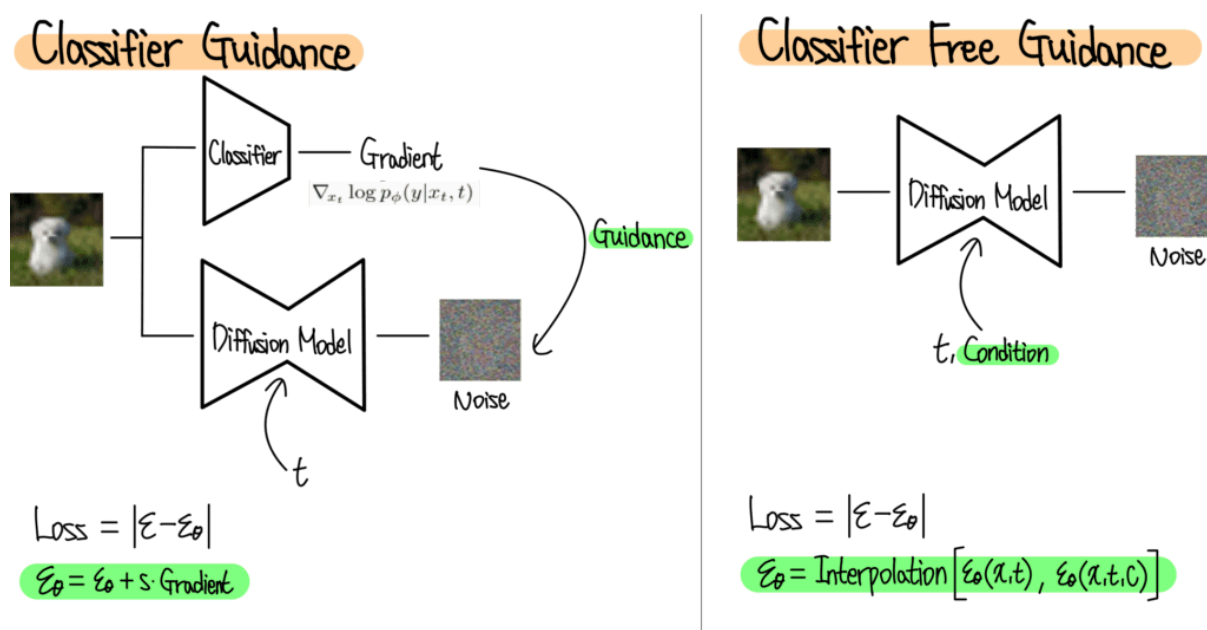


그림14. Classifier Free Guidance

위 그림은 Classifier Guidance와 Classifier Free Guidance 방법을 비교하고 있는데요. 우측의 Classifier Free Guidance 그림을 보면 Condition을 입력 받아 예측한 Noise와 Condition 없이 예측한 Noise의 Interpolation을 최종 Noise로 사용하는 모습을 볼 수 있습니다. 자세한 내용은 [Classifier Free Guidance 설명글](#)을 참고해 주세요.

3-2. Super-Resolution

다음으로 Diffusion Model은 Super-Resolution을 잘 할 수 있습니다. Super-Resolution이란 저화질 이미지를 고화질 이미지로 변환하는 작업을 말합니다. 이는 대표적으로 2023년 PAML에 발표된 SR3의 방법을 들 수 있는데요. 방법은 다음과 같습니다.

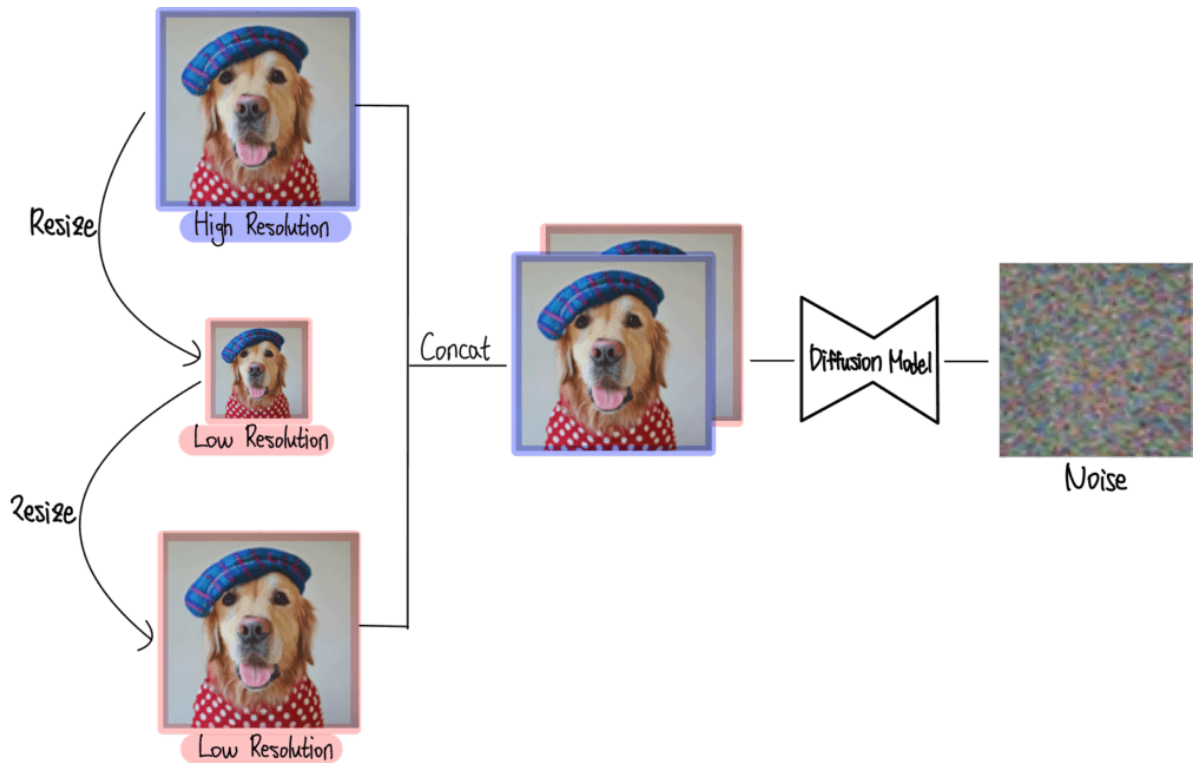


그림15. Super-Resolution

위 그림을 보면 Diffusion Model의 입력으로 High Resolution 이미지와 Low Resolution 이미지가 들어가는 모습을 볼 수 있는데요. 이렇게 학습한 Diffusion 모델은 Low Resolution 이미지만을 받아서 High Resolution 이미지를 생성할 수 있게 되죠. 이렇게 SR3 모델의 Super-Resolution 결과 샘플은 다음과 같습니다.

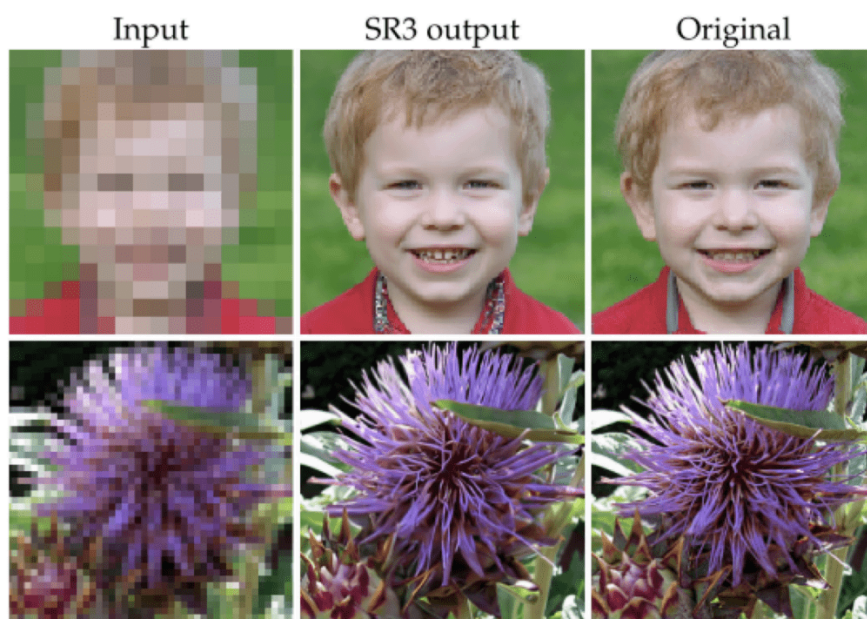


그림16. SR3 Super-Resolution 결과

Low Resolution 이미지를 입력으로 받아 거의 원본과 가까운 High Resolution 이미지를 잘 생성해냈죠? 자세한 내용은 [SR3 설명글](#)을 참고해 주세요.

3-3. Inpainting

다음으로 Diffusion Model은 Inpainting도 할 수 있습니다. Inpainting이란 이미지의 가려진 부분을 그럴듯하게 채워 넣는 기술을 말합니다. 이를 위한 Diffusion Model은 다음과 같이 구성해줍니다.

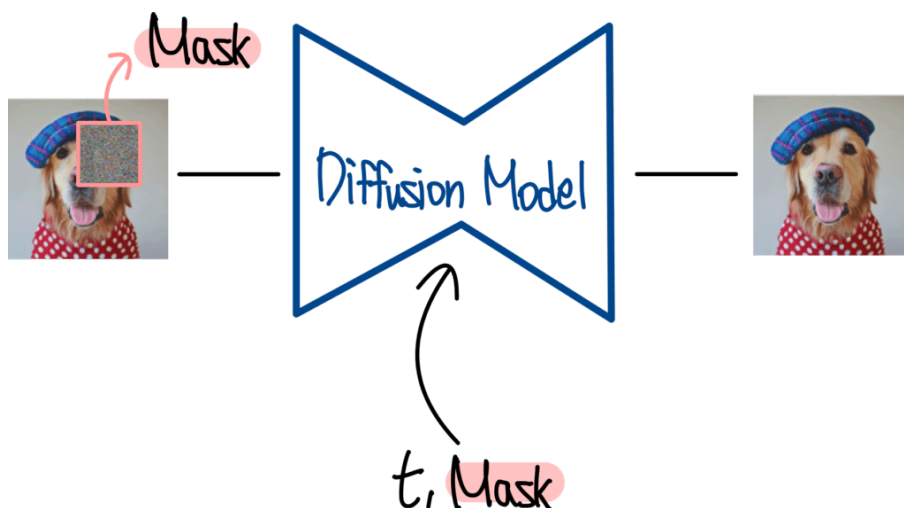


그림17. Inpainting Diffusion Model

위 그림을 보면 Diffusion Model의 입력으로 Mask 처리된 이미지가 들어가는 모습을 볼 수 있습니다. 또한 이때의 Mask 정보는 Condition으로 주어지죠. 그럼 Diffusion 모델은 Mask 처리된 부분의 원래 모습을 예측하도록 학습해주는 겁니다. 이렇게 학습이 완료된 모델에게 Mask 처리된 다른 이미지와 Mask 정보를 주면 그럴듯하게 Mask 부분을 채워서 출력해주는 구조입니다.

3-4. Text Guided Image Generation

다음은 Text가 묘사하는 이미지를 생성하는 Diffusion Model들을 살펴보겠습니다. 이른바 Text Guided Image Generation 인데요. 가장 대표적인 모델 세 가지의 핵심 내용을 살펴 보겠습니다.

3-4-1. GLIDE

먼저 OpenAI에서 발표한 [GLIDE](#)라는 모델입니다. GLIDE의 구조는 다음과 같습니다.

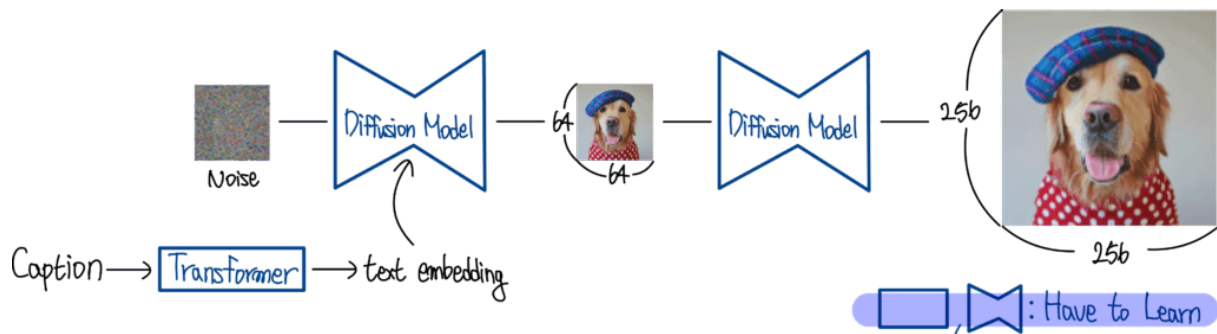


그림18. GLIDE Architecture

크게 보면 Text Embedding을 학습하는 Transformer와 이미지 생성을 위한 Diffusion Model, 그리고 Super-Resolution을 위한 Diffusion Model로 구성되어 있죠. 이후 발표된 모델들과의 결정적인 차이라고 한다면 Text Embedding을 학습하기 위한 Transformer라고 할 수 있습니다. 이후 모델들은 Text Embedding을 이렇게 따로 학습하지 않고 대규모 언어 데이터로 학습한 Pretrained Text Encoder를 사용하거든요. 이후 모델들보다 성능은 조금 아쉽지만 GLIDE가 생성한 이미지들을 살펴보면 다음과 같습니다.



"a hedgehog using a calculator"



"a corgi wearing a red bowtie and a purple party hat"



"robots meditating in a vipassana retreat"



"a fall landscape with a small cottage next to a lake"



"a surrealist dream-like oil painting by salvador dali of a cat playing checkers"



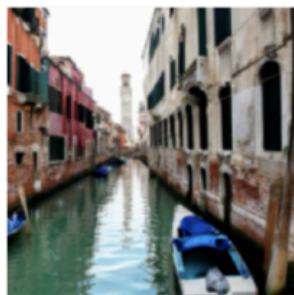
"a professional photo of a sunset behind the grand canyon"



"a high-quality oil painting of a psychedelic hamster dragon"



"an illustration of albert einstein wearing a superhero costume"



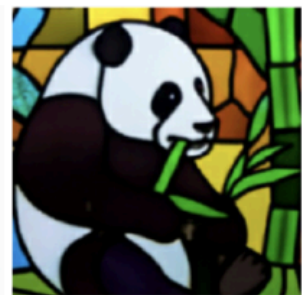
"a boat in the canals of venice"



"a painting of a fox in the style of starry night"



"a red cube on top of a blue cube"



"a stained glass window of a panda eating bamboo"



"a crayon drawing of a space elevator"



"a futuristic city in synthwave style"



"a pixel art corgi pizza"



"a fog rolling into new york"

그림19. GLIDE가 생성한 이미지

이미지를 묘사하는 Caption에 맞게 적절한 이미지들을 생성해주는 모습이죠? GLIDE에 대한 자세한 내용은 [GLIDE 설명글](#)을 참고해주세요.

3-4-2. Imagen

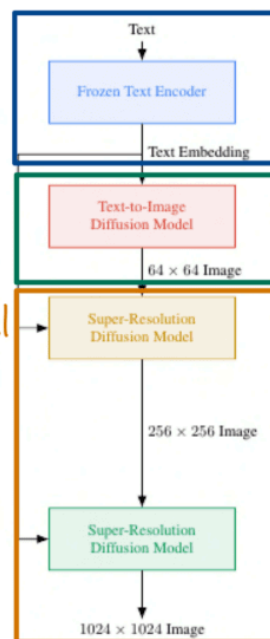
다음은 Google Research에서 발표한 Imagen을 살펴보겠습니다. Imagen은 GLIDE에 비해 아주 간단한 구조를 하고 있습니다.

1. Pretrained Text Encoder

2. Classifier Free Guidance

3. Weight Sampler

4. Cascaded Diffusion Model



"A Golden Retriever dog wearing a blue checkered beret and red dotted turtleneck."



그림20. Imagen Architecture

위 그림은 Imagen Architecture를 표현한 그림인데요. GLIDE와 거의 비슷한 구조를 하고 있죠. 먼저 Caption의 Text Embedding을 추출해주고 Diffusion Model이 이미지를 생성한 뒤 이어서 Super Resolution까지 직렬로 연결되어 있습니다. GLIDE와 비슷한 구조를 하고 있지만 Pretrained Text Encoder를 사용했다는 점과 더 고화질의 Super-Resolution을 수행했다는 차이를 갖고 있습니다. 이렇게 학습한 Imagen 모델이 생성한 이미지들은 다음과 같습니다.



Sprouts in the shape of text 'Imagen' coming out of a fairytale book.



A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.



A high contrast portrait of a very happy fuzzy panda dressed as a chef in a high end kitchen making dough. There is a painting of flowers on the wall behind him.



Teddy bears swimming at the Olympics 400m Butterfly event.



A cute corgi lives in a house made out of sushi.



A cute sloth holding a small treasure chest. A bright golden glow is coming from the chest.



A brain riding a rocketship heading towards the moon.



A dragon fruit wearing karate belt in the snow.



A strawberry mug filled with white sesame seeds. The mug is floating in a dark chocolate sea.

그림21. Imagen이 생성한 이미지

이젠 정말 엄청난 이미지들을 만들어주고 있죠? 자세한 내용은 [Imagen 설명글](#)을 참고해주세요.

3-4-3. DALLE2

다음은 또다시 OpenAI에서 발표한 [DALLE2](#) 모델입니다. DALLE2는 OpenAI에서 발표한 CLIP을 아주 적절하게 활용해주는데요. 전체 구조는 다음과 같습니다.

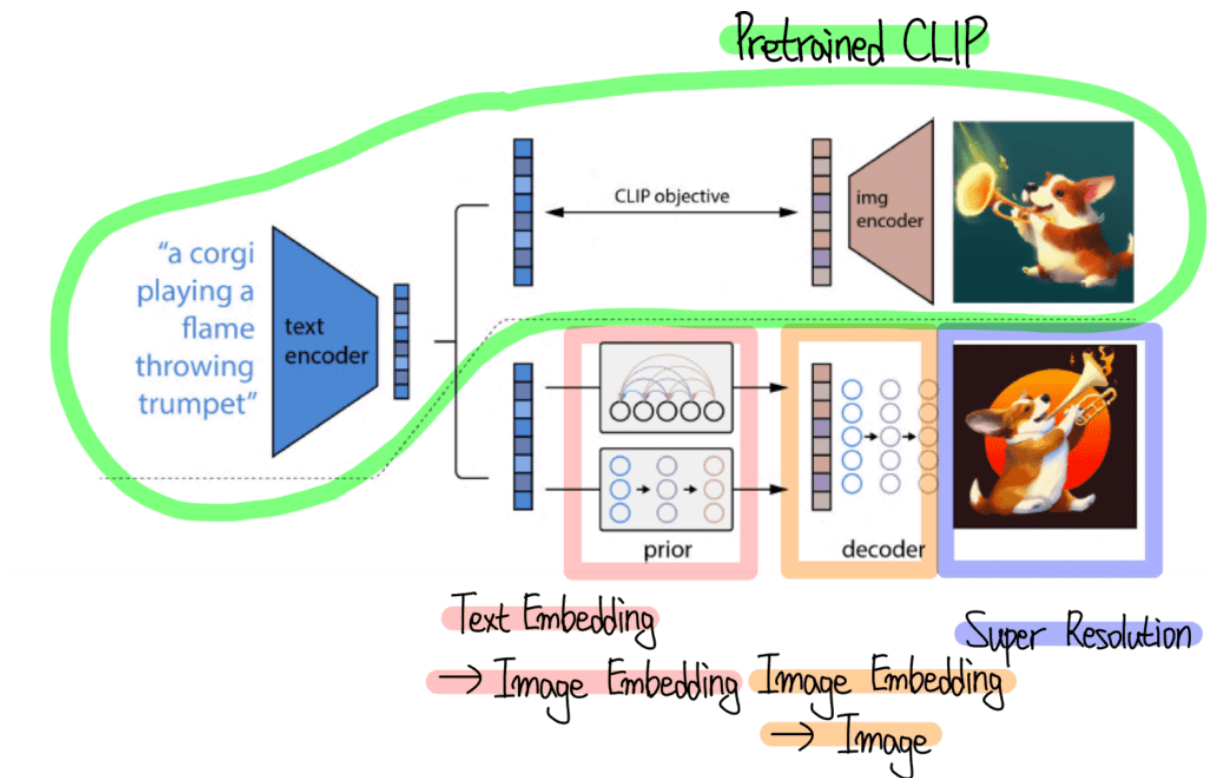
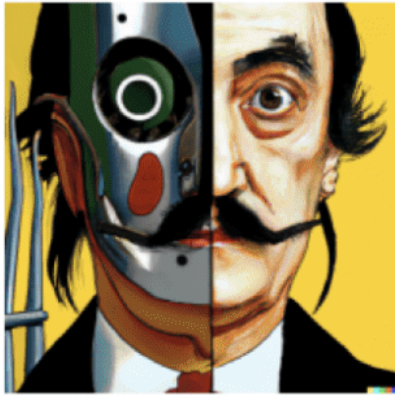


그림22. DALL-E2 Architecture

가장 위쪽에서는 Pretrained CLIP을 사용하여 Text와 Image의 Embedding을 추출하는 모습을 표현하고 있습니다. 그 아래 Prior와 Decoder로 표현되어 있는 Diffusion Model에서는 각각 Image Embedding과 이미지를 생성하도록 학습하는 모습을 표현하고 있습니다. 이렇게 학습한 DALL-E2가 생성한 이미지들은 다음과 같습니다.



a vibrant portrait painting of Salvador Dali with a robotic half face



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it



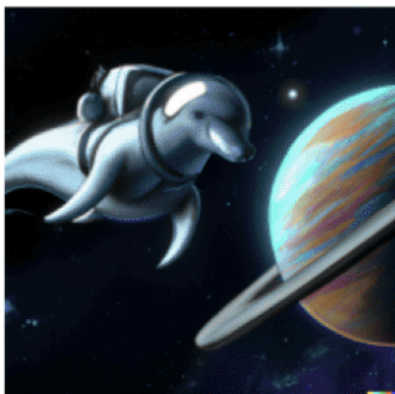
an espresso machine that makes coffee from human souls, artstation



panda mad scientist mixing sparkling chemicals, artstation



a corgi's head depicted as an explosion of a nebula



a dolphin in an astronaut suit on saturn, artstation



a propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese



a teddy bear on a skateboard in times square

그림23. DALLE2가 생성한 이미지

복잡한 Caption도 그럴듯하게 고화질로 이미지를 생성해주는 모습이죠?

재미있는건 DALLE2에서는 이미지 생성뿐만 아니라 이미지 조작 (Manipulation) 방법도 제안합니다. 예를 들어 서로 다른 두 이미지를 적절하게 섞어줄 수 있는데요. 이렇게요.

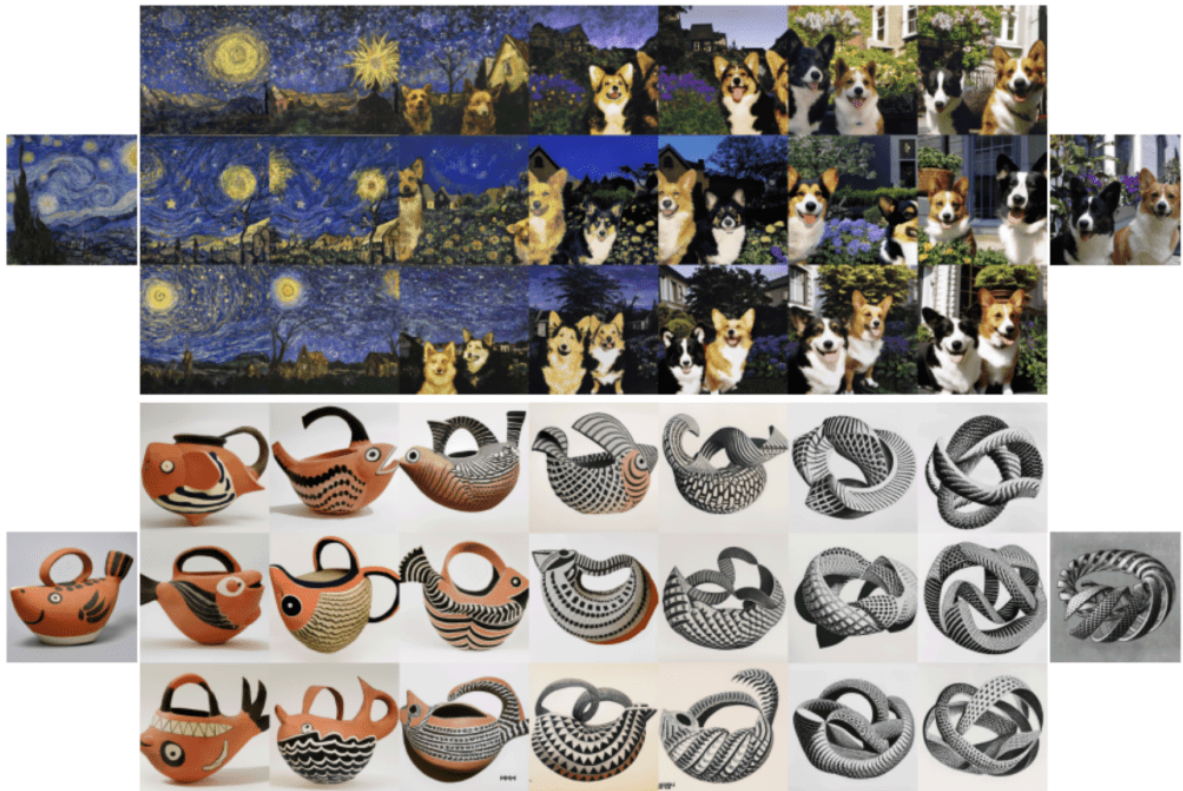


그림24. DALLE2 Image Manipulation

왼쪽 이미지와 오른쪽 이미지를 적절하게 잘 섞어 가운데 이미지들을 생성한 모습을 볼 수 있습니다. DALLE2에서는 이뿐만 아니라 다양한 방법의 Manipulation 을 제안하고 있습니다. 자세한 내용은 [DALLE2 설명글](#)을 참고해주세요.

3-5. Multi Modal Guided Image Generation

GLIDE, Imagen, DALLE2 등 엄청난 성능의 Diffusion Model들을 살펴봤는데요. 하지만 이들 모델들에도 공통적인 한계가 존재합니다. 바로 Condition으로 Text만 입력받을 수 있다는 것이죠. 예를 들어 Bounding Box를 Condition으로 입력받아 어느 위치에는 사람을, 어느 위치에는 자동차를 그릴 수는 없을까요? 이러한 아이디어를 구현한 모델로는 [Stable Diffusion](#)이 있습니다.

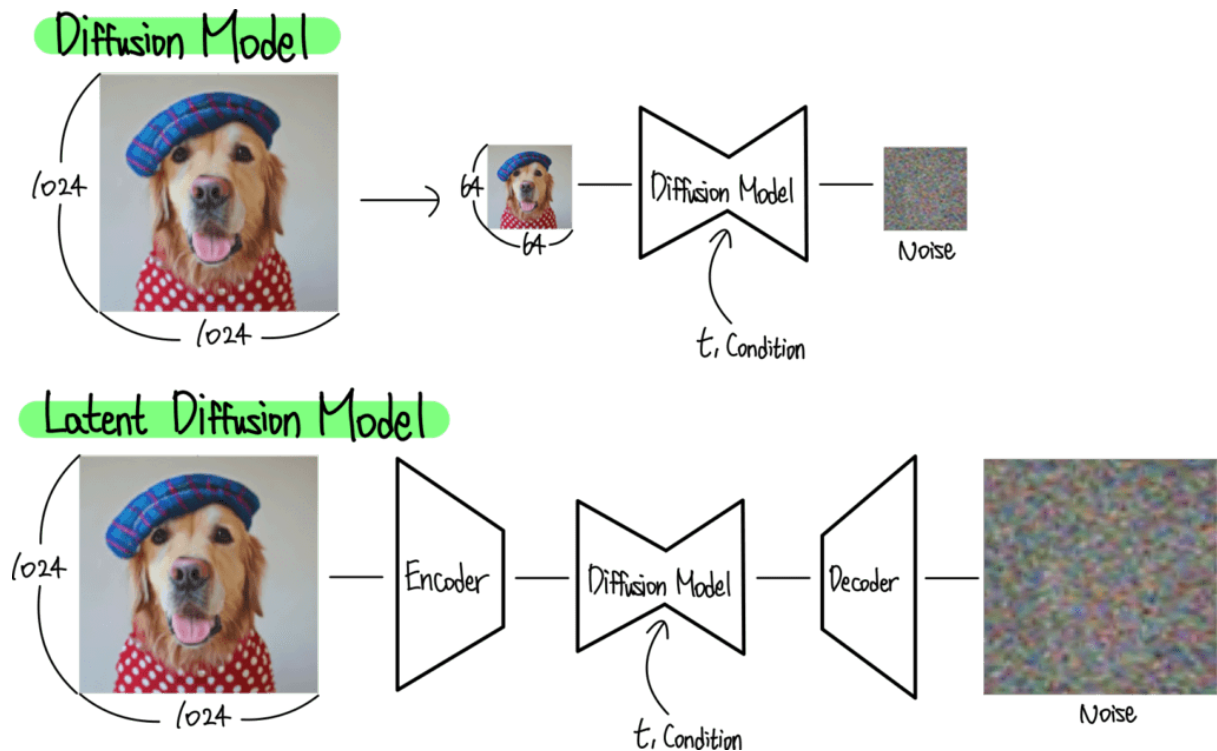


그림25. Stable Diffusion Architecture

위 그림은 기존 Diffusion Model과 Stable (Latent) Diffusion Model의 차이를 비교하고 있는데요. Stable Diffusion Model은 Auto Encoder를 사용하여 이미지를 압축 한 뒤 Diffusion Model을 학습해 줍니다. 덕분에 계산량도 많이 줄고 Semantic한 부분에 더 초점을 맞출 수 있게 되었죠.

이렇게 학습한 Stable Diffusion Model이 생성한 그림들은 다음과 같습니다. 먼저 Text To Image 결과입니다.

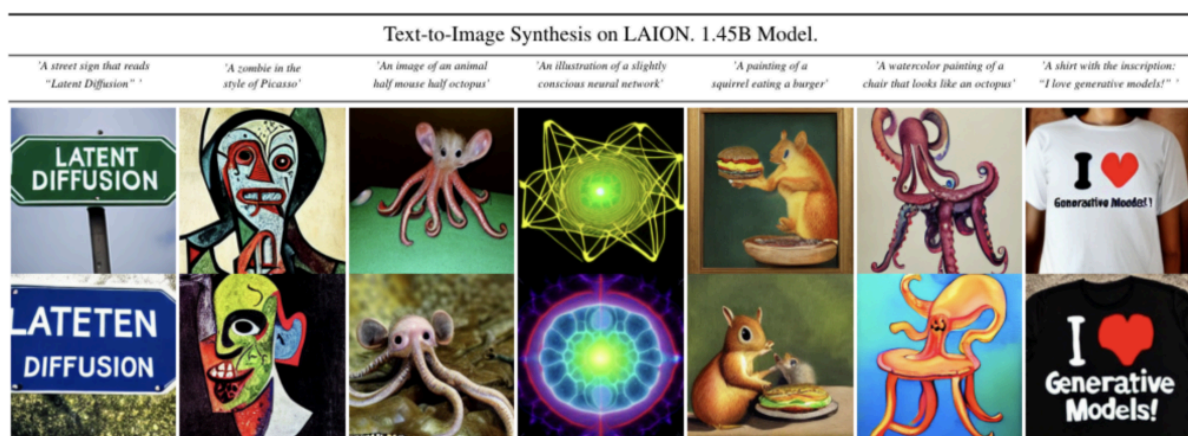


그림26. Stable Diffusion Text To Image Result

이는 위에서 봤던 GLIDE, Imagen, DALLE2의 결과와 비슷하죠. 다음은 Layout To Image 결과입니다.



그림27. Stable Diffusion Layout To Image Result

왼쪽의 Layout 정보를 입력 받아 위치에 맞는 이미지들을 만들어주는 모습을 볼 수 있습니다. 자세한 내용은 [Stable Diffusion 설명글](#)을 참고해주세요.

3-6. Multi Modal Input Image Generation

이렇게 엄청난 Stable Diffusion에도 한계가 존재하는데요. 다양한 형태의 Condition을 입력으로 받을 수 있지만, 하나만 선택해야 한다는 점입니다. 예를 들어 Text와 Layout을 동시에 입력으로 받을 수는 없죠. 게다가 Diffusion Model을 처음부터 학습하기 위해서는 엄청난 Computing Resource를 필요로 합니다. 이 두 가지 한계를 모두 극복한 모델이 바로 [GLIGEN](#) 인데요. GLIGEN의 구조는 다음과 같습니다.

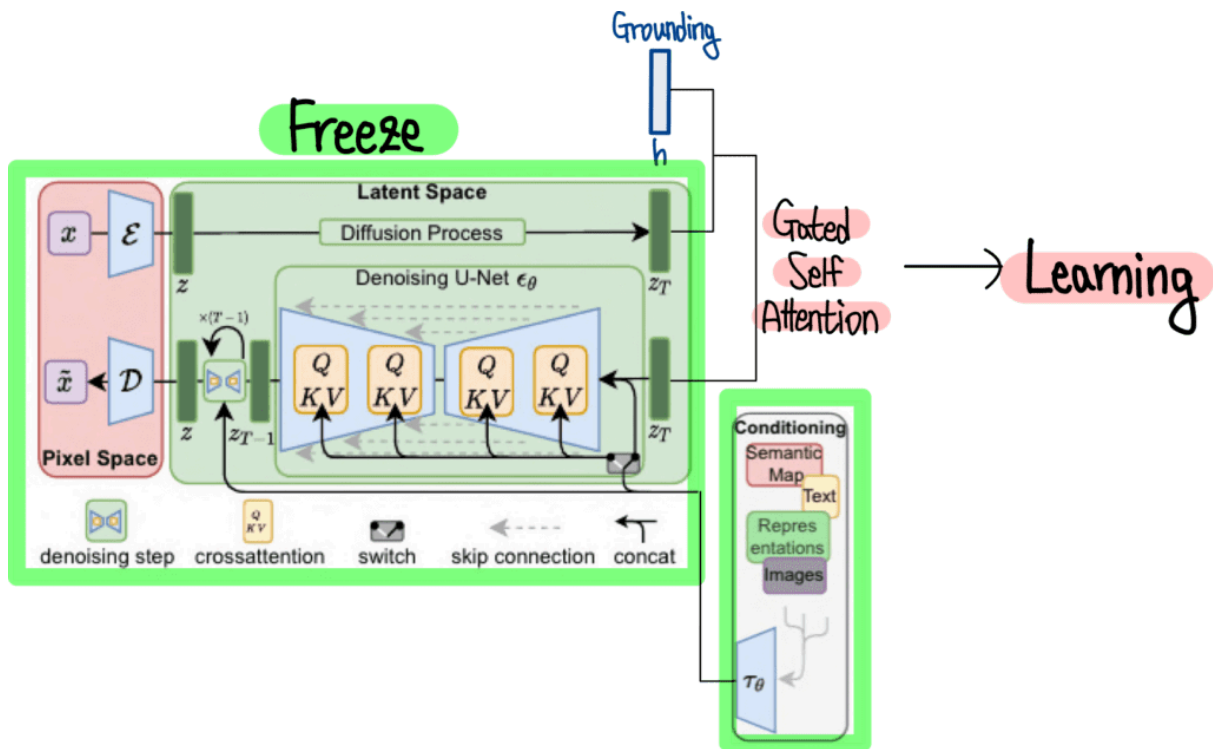


그림28. GLIGEN Architecture

몸통 부분은 Stable Diffusion이 그대로 들어있는 모습이죠. 이 외에 Grounding 정보를 연산하기 위한 Gated Self Attention 부분만 추가되었는데요. Grounding은 Layout과 같이 Text 외의 추가 정보를 의미합니다. GLIGEN은 Stable Diffusion 부분은 Freeze 한 채로 Gated Self Attention 부분만 학습해줍니다.

이렇게 학습한 GLIGEN이 생성한 이미지들은 다음과 같습니다.

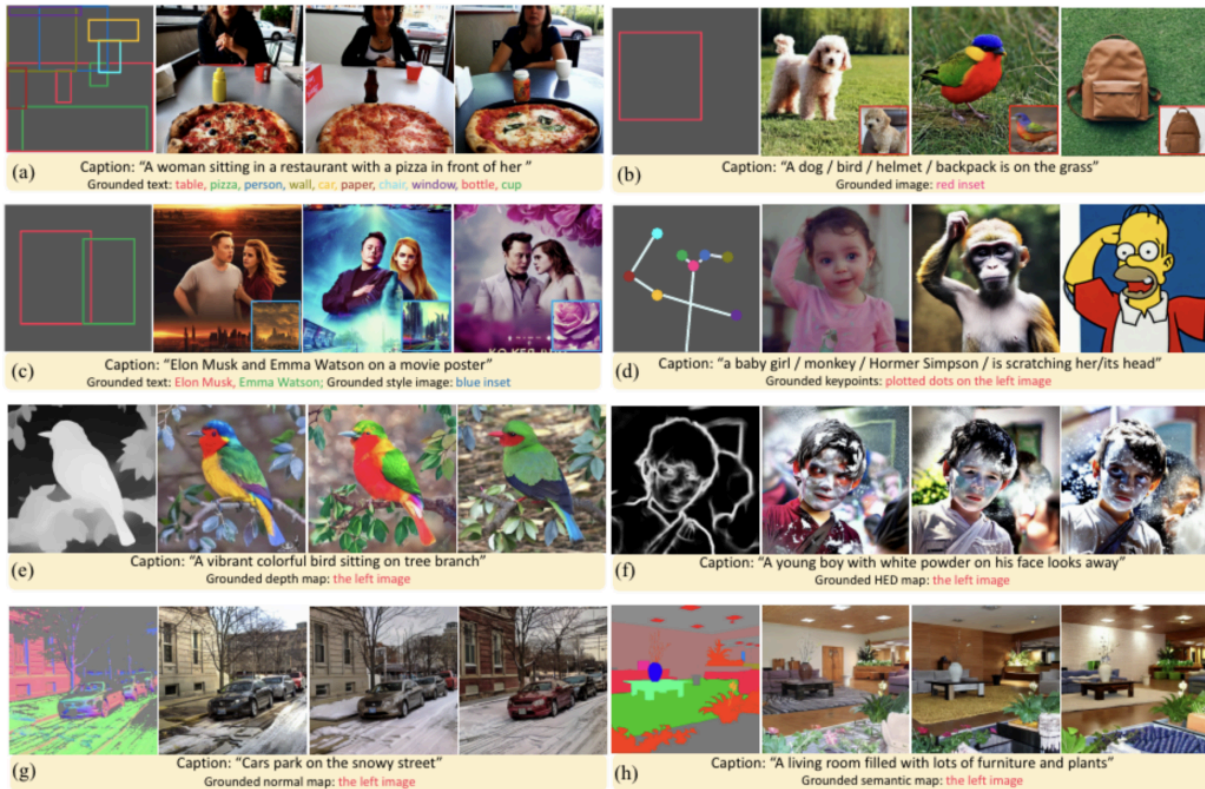


그림29. GLIGEN이 생성한 이미지

Caption과 다양한 형태의 Grounding을 입력 받아 이미지를 생성하고 있는 모습을 볼 수 있는데요. Key Points, Depth Map, HED Map, Normal Map, Semantic Map 등 다양한 형태의 Grounding을 입력 받는 모습을 볼 수 있습니다. 자세한 내용은 [GLIGEN 설명글](#)을 참고 바랍니다.

4. 마치며

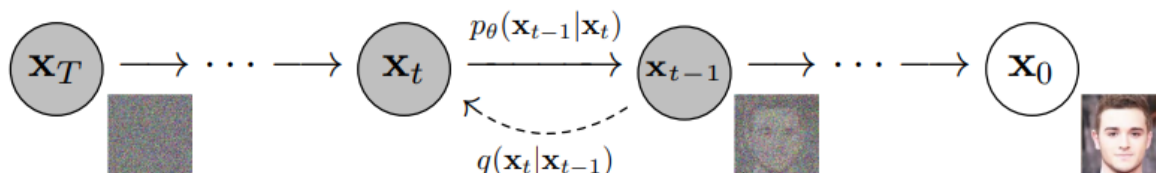
지금까지 Diffusion Model의 기초부터 응용까지 살펴보았습니다. 기초 부분에서는 Diffusion Model이 무엇인지, 어떻게 동작하는지, Architecture와 Loss Function에 대해 살펴보았습니다. 응용 부분에서는 단순한 이미지 생성을 넘어 다양한 조건하에서 이미지를 생성하는 모델들을 살펴보았습니다.

Diffusion Model은 요즘 가장 큰 관심을 받고 있는 주제 중 하나입니다. 언어 모델 분야에서는 ChatGPT 등이 사람을 능가하는 능력을 보여주어 화제가 되고 있다면, 이미지 모델 분야에서는 Diffusion Model이 사람을 넘어서는 능력을 보여주어 화제가 되고 있죠. 최근에는 이미지 생성을 넘어 동영상 생성까지 활발히 연구되고 있습니다.

Diffusion model은 데이터를 만들어내는 deep generative model 중 하나로, **data로부터 noise를 조금씩 더해가면서 data를 완전한 noise로 만드는 forward process(diffusion process)와 이와 반대로 noise로부터 조금씩 복원해가면서 data를 만들어내는 reverse process**를 활용한다. Diffusion model은 현재 image 쪽에서는 가장 잘 작동하는 generative model 중 하나이기 때문에 기본으로 알아놓으면 아주 유용할 것 같다. 그러므로 이번 글에서는 diffusion model이 무엇인지에 대해 알아보겠다.

Overview

Diffusion model은 위에서도 언급한 것처럼, data에 noise를 조금씩 더해가거나 noise로부터 조금씩 복원해가는 과정을 통해 data를 generate하는 모델이다. 이를 한 눈에 표현하면 아래 그림과 같다. 아래 그림에서 x_0 는 실제 데이터, x_T 는 최종 noise, 그리고 그 사이의 x_{t-1} 는 데이터에 noise가 더해진 상태의 latent variable을 의미한다.



Diffusion model

우선, 위 그림의 오른쪽에서 왼쪽 방향으로 **noise를 점점 더해가는 forward process q** 를 진행한다. 그리고 이 **forward process**를 반대로 추정하는 **reverse process p** 를 학습함으로써 **noise(x_T)로부터 data(x_0)를 복원하는 과정을 학습한다**. 그리고 이 reverse process를 활용해서 **random noise로부터 우리가 원하는 image, text, graph 등을 generate할 수 있는 모델을 만들어내는 것이다**. 그리고 이를 통해 결국, 우리는 실제 data의 분포인 $p(x_0)$ 를 찾아내는 것을 목적으로 한다. 그럼 이를 좀 더 자세히 살펴보겠다.

Reverse process

Reverse process p 는 **noise(x_T)로부터 data(x_0)를 복원하는 과정**이다. 이는 최종적으로 random noise로부터 data를 generate하는 generative model로 사용되기 때문에 diffusion model을 사용하기 위해서는 모델링하는 것이 필수적이지만, 이를 실제로 알아내는 것은 쉽지 않다. 그러므로 우리는 p_θ 를 활용해서 이를 approximate한다. 이 때, 이 approximation은 Gaussian transition을 활용한 Markov chain의 형태를 가진다. 이를 식으로 표현하면 다음과 같다.

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

reverse process의 수식 표현

위 식에서, 각 단계의 정규 분포의 평균 μ_θ 와 표준편차 Σ_θ 는 학습되어야 하는 parameter 들이다. 그리고 위 식의 시작지점인 noise의 분포는 다음과 같이 가장 간단한 형태의 표준정규분포로 정의한다.

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

noise의 분포

Forward process

Forward process q 는 **data**($\mathbf{x}_0$)으로부터 **noise**를 더해가면서 최종 **noise** ($\mathbf{x}_T$) 형태로 가는 과정이다. 우리가 이 과정의 분포를 알아내야 하는 이유는, reverse process의 학습을 forward process의 정보를 활용해서 하기 때문이다. Forward process는 reverse process와 비슷하지만 약간 다르게, data에 Gaussian noise를 조금씩 더하는 Markov chain의 형태를 가진다. 이를 식으로 표현하면 다음과 같다.

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

forward process의 수식 표현

이제부터 약간의 수식 파티를 할 예정인데, 그냥 diffusion model이 이런거구나~ 하고 넘어가실 분들은 아래 내용은 읽지 않아도 좋지만 읽으면 더 좋다. 위와 같은 형태로 forward process를 표현할 때의 장점은 data가 주어졌을 때 임의의 timestep t 시점에서의 $\mathbf{x}_t$ 를 자유롭게 sampling할 수 있다는 점이다. 어떤 data $\mathbf{x}_0$ 가 주어졌을 때 $\mathbf{x}_t$ 의 분포는 다음과 같다.

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

$\mathbf{x}_0$ 가 주어졌을 때 $\mathbf{x}_t$ 의 분포

위 식에서 α 는 다음과 같은 식을 따른다.

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

이의 증명은 다음과 같다. (증명은 생략해도 좋다.)

Training

$$\mathbb{E} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

diffusion model의 training loss

$$\begin{aligned} -\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) \| p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\ &= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})/p_\theta(\mathbf{x}_0)} \right] \\ &= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0) \right] \\ &= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \end{aligned}$$

위 식의 두 번째 부등호의 증명. 마지막 식에서 분모와 분자를 바꾸고 -를 붙이면 위 식과 같다.

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t \geq 1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

diffusion model의 최종적인 training loss

- L_T : p 가 generate하는 noise $\mathbf{x}_T$ 와 q 가 $\mathbf{x}_0$ 라는 데이터가 주어졌을 때 generate하는 noise $\mathbf{x}_T$ 간의 분포 차이
- L_{t-1} : p 와 q 의 reverse / forward process의 분포 차이. 이들을 최대한 비슷한 방향으로 학습한다.
- L_0 : latent $\mathbf{x}_1$ 으로부터 data $\mathbf{x}_0$ 를 추정하는 likelihood. 이를 maximize하는 방향으로 학습한다.

이에 대한 증명은 다음과 같다.

$$\begin{aligned}
L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \\
&= \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \left(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} \right) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right] \\
&= \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))}_{\widetilde{L}_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{\widetilde{L}_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{\widetilde{L}_0} \right]
\end{aligned}$$

diffusion model의 최종 training loss 증명

결론적으로, 우리가 diffusion model을 학습할 때의 training loss는 정규 분포 사이의 KL divergence 형태로 쉽게 계산될 수 있는 것이다.